Hobart Corporation



HLX Scale

Software Architecture Document

10/04/2010

Revision: 1.0

Submitted by:

BSQUARE Corporation
110 110th Ave NE Suite 200
Bellevue, WA 98004
(425) 519-5900 (Voice)
(425) 519-5999 (Fax)

To:

Hobart Corporation
701 S. Ridge Ave
Troy, OH 45374

Attn: Debra Dyer
Telephone #

IMPORTANT: This document is intended only for the use of the individual or entity to which it is addressed, and contains information, which is confidential and proprietary. If the reader of this document is not the intended recipient, or the employee or agent responsible for delivering the document to the intended recipient, you are hereby notified that any dissemination, distribution or copying of this document is strictly prohibited. If you have received this document in error, please notify the sender immediately and return the original document.

Project Name	HLX Scale Linux Port
Document ID	
Author(s)	Minhaj Ahmed Sr. Software Engineer II Bsquare Professional Engineering Services (PES) 2100 Lakeside Blvd. St. 450 Richardson, TX 75082
Reviewed by	Michael Browne Director of Engineering Bsquare Professional Engineering Services (PES) 2100 Lakeside Blvd. St. 450 Richardson, TX 75082
Approved by:	Bsquare Project Manager: John Imboden Sr. Program Manager 3480 West Market Street, suite 105 Fairlawn, OH 44333 Client Project Manager: Name Title Division
Delivery Date	

Revision History

Revision	Date	By	Comments
0.1	10/04/2010	M. Ahmed	Initial Draft
1.0	10/13/2010	M. Ahmed	Initial Release

Contents

1 PORTING METHODOLOGY 1
1.1 OVERVIEW 1
1.2 SOFTWARE PORTING 1
1.2.1 Non-Portable Software 1
1.2.2 Largely Portable Software 3
1.2.3 Non-standard code and Compiler incompatibilities 8
1.3 BUILD SYSTEM 8
1.4 ASSUMPTIONS 12

XPe	Windows XP Embedded
OS	Operating System
UI	User Interface
API	Application Programmer's Interface
Abstract Class	A class which cannot be instantiated and serves as an interface only.
Pure Virtual	Methods of an abstract class which serve only as an interface for the derived classes. Abstract class itself does not necessarily have an implementation for these.
Sync Objects	Synchronization primitives like mutex, Critical Sections etc.

1 Porting Methodology

1.1 Overview

Generally, there are three main categories of software that need to be considered when porting an application from one operating system to another. The first category is intrinsically non-portable software that includes, for example, drivers, services and service installers. Non-portable software works very closely with the platform specific features and software APIs. The second category is largely portable software that has sparse usage of platform specific APIs and features. The third category consists of software which is incompatible due to either the usage of non-standard APIs or the differences between compilers for given platforms. This software category requires special attention during porting.

Hobart's HLX product application software consists of several components which include one or more of the aforementioned three categories of software. The remainder of this document details the strategy used and the assumptions made in order for the HLX application software to run under Microsoft Windows and Linux.

1.2 Software porting

This section documents the methodology which was followed to handle each of the three categories of ported software.

1.2.1 Non-Portable Software

Since this category of software is intrinsically non-portable, it had to be rewritten for the Linux platform. However, in order to use the same code base on both platforms (Windows and Linux) *build time conditional compilation* was used to isolate the non-portable software components and to build only the platform specific files. Additionally, since Microsoft Visual Studio supports XML based project files exclusively, it was not possible to use the same build scripts for both Windows and Linux. Therefore, the final build contains both make files and .vcproj files that each includes appropriate platform specific and ported files.

Project components included in this category are:

- appInstaller
- winappLauncher
- WeighUsb Drivers
- CNTService

Also included in this category are classes from the device interface section of the application which are heavily dependent on Windows APIs. In order to maintain the readability of the code it was necessary to separate these classes into platform specific files. *UsbManager* and *UsbNode* are examples of two such classes.

The approach taken to achieve this goal has been detailed in Figures 1.1, 1.2 and 1.3. UsbManager was implemented as a singleton class with global method *getInstance* providing the necessary interface for accessing the class instance. In order to appropriately separate platform specific functionality, *getInstance* method was modified to return the instance of platform specific derived class (Figure 1.3). Additionally, several platform specific methods of *UsbManager* class were declared *pure virtual* transforming *UsbManager* into an *Abstract class*. Linux and Windows specific classes were then derived from this abstract class to handle platform specific details. This modification neatly allowed the platform specific implementation to be moved to independent classes while keeping all the rest of the code intact. It also ensured that any future platform support would simply require an additional class to be added to the software application without the need of any further modifications to the base class.

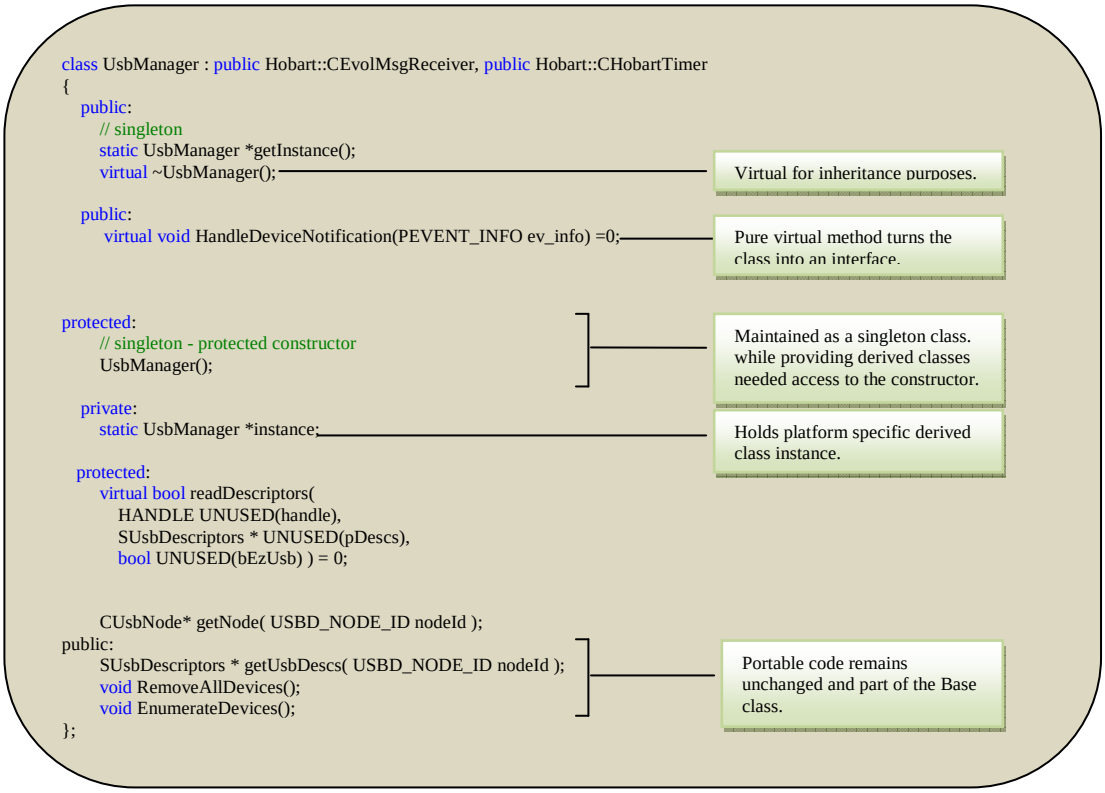


Figure 1.1 Important sections of UsbManager Base class

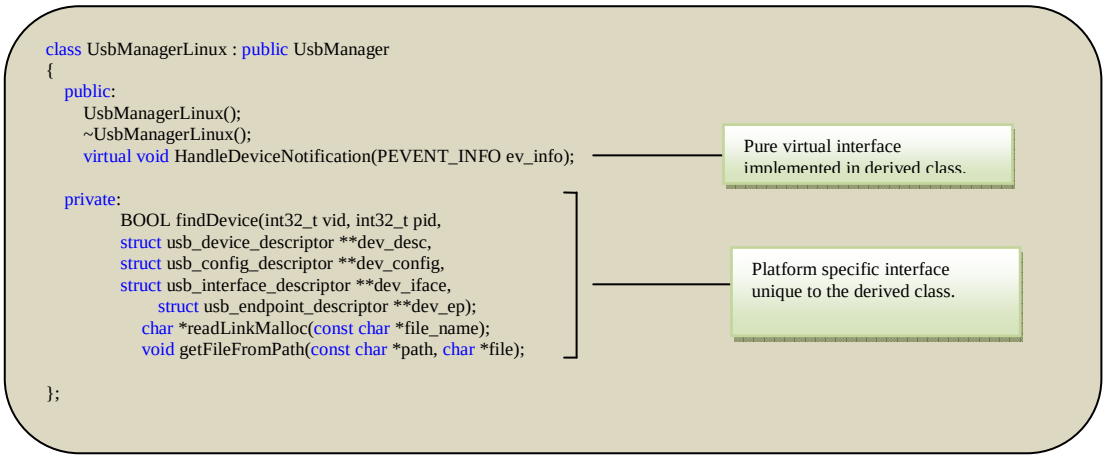


Figure 1.2 Important sections of UsbManager derived class UsbManagerLinux.

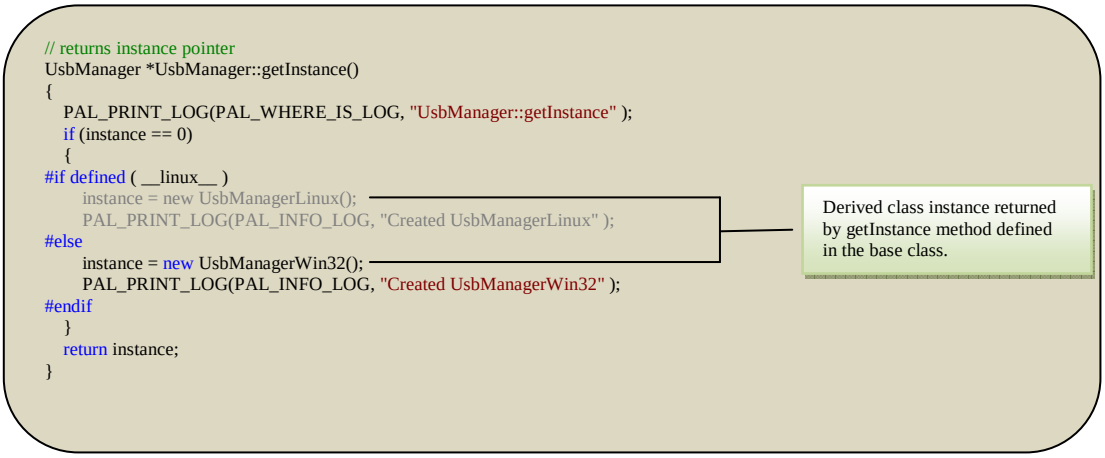


Figure 1.3 UsbManager::getInstance() method

1.2.2 Largely Portable Software

Large portion of the HLX application were implemented using either standard C++ or APIs which have standard C++ equivalents on both operating systems. This enabled majority of the application software to be free from any C++ compiler specific incompatibilities. The HLX application also implements classes for data serialization. These classes are useful for storing data in network buffers as well as making software independent of the underlying processor. This section however details non-portability solutions at the software level only and assumes that there would be no changes to the underline hardware.

PAL (Platform Abstraction Layer) uses following three abstraction methods to introduce portability to software falling in this category

- Bridge Routines
- Link Time Resolution
- C Preprocessor

In order to make the HLX application completely independent of platform specific calls, classes exporting *bridge routines* are introduced. These bridge routines act as an abstraction layer providing a platform independent interface to the HLX application while acting as wrapper for platform specific functionality. *Link time resolution* is used to link-in the platform specific version of the APIs. This concept is similar to Microsoft's implementation of HAL (Hardware Abstraction Layer), where the software using the bridge routines is agnostic to the differences in the underlying platform depending entirely on the bridge routines to handle all the platform specific details (see Figure 1.4). The appropriate platform specific bridge routines are linked in at build time allowing the portable software to work seamlessly.

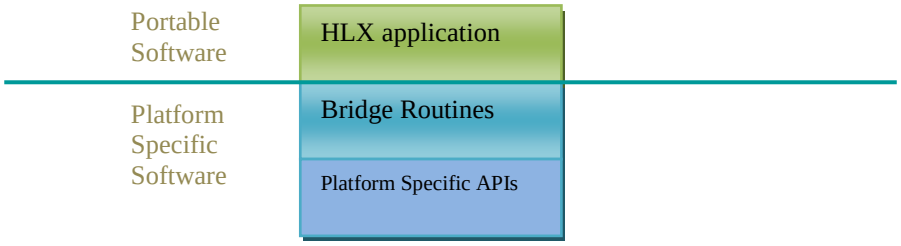


Figure 1.4 Bridge Routine Abstractions

One of the primary goals while porting the HLX application was to maintain the readability of the software for developers familiar with Windows programming by moving all portability details into PAL and supporting headers. While this approach maintained the readability of the code it introduced the challenge of handling input and output parameter differences in the underlying platform specific APIs. In order to handle these differences, two approaches were taken. First, PAL user defined types, equivalent to Windows return types, were declared for all return values for various APIs. Secondly, Linux specific API error codes were mapped to corresponding Window's error codes. This approach allowed the HLX application to maintain the same code flow and error handling logic across Windows and Linux.

For input parameters however, Bsquare maintained the structure of Windows APIs and implemented bridge methods which accept all Windows parameters. Any unused parameter was then marked as such using compiler directives. This approach allowed PAL APIs to receive and return the same parameters regardless of the platform and this approach limited the number of changes required in the original code, maintaining the readability for Windows developers. Figures 1.5 and 1.6 detail this approach.

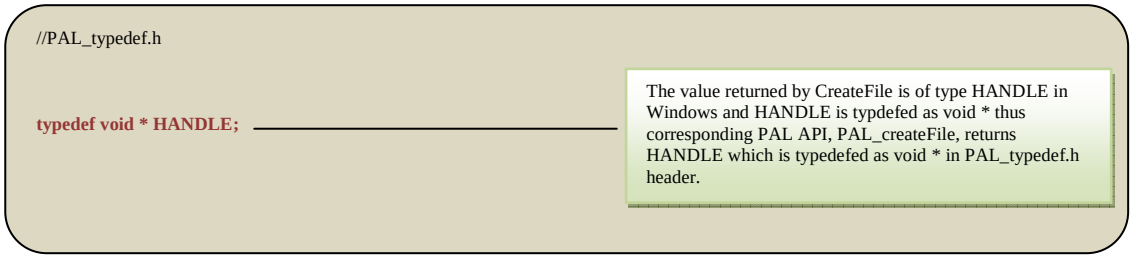


Figure 1.5 Usage of user defined types-unmatched parameters

```
static HANDLE   PAL_createFile(
                ///[in] The name of the file or device to be created or opened.
                LPCSTR name,
                ///[in] The requested access to the file or device.
                DWORD access_right,
                ///[in] The requested sharing mode of the file or device.
                DWORD share_mode,
#if defined ( __linux__ )
                ///[in] UNUSED-A pointer to a SECURITY_ATTRIBUTES structure.
#else
                ///[in] A pointer to a SECURITY_ATTRIBUTES structure.
#endif
                LPSECURITY_ATTRIBUTES secure_attr_p,
                ///[in] An action to take on a file or device that exists or does not exist.
                DWORD disposition,
                ///[in] The file or device attributes and flags.
                DWORD flag_and_attribute,
#if defined ( __linux__ )
                ///[in] UNUSED-A valid handle to a template file with the GENERIC_READ
                ///access right.
#else
                ///[in] A valid handle to a template file with the GENERIC_READ access
                ///right.
#endif
                ///The template file supplies file attributes and extended attributes
                ///for the file that is being created.
                HANDLE template_file
                    );
```

Figure 1.6 PAL_createFile declaration in PAL.

The combination of bridge routines and link time resolution works in most cases. However, in some circumstances it is not possible to cleanly resolve differences between the platform APIs and features by using this approach. In such cases a C pre-processor is invoked to isolate platform specific dependencies. Even though usage of pre-processor is simple and powerful, over usage can affect code readability considerably and for this reason its usage was limited.

Another area where C pre-processor is required is where a given routine has differs in the number of arguments accepted which cannot be easily resolved or where Windows uses a slightly modified name for a standard routine. In such cases PAL uses *#define* to redirect the call appropriately for the given platform (Figure 1.6)

```
#define _strtod_l          strtod_l
#define _strftime_l        strftime_l
#define _locale_t          locale_t
#define sscanf_s           sscanf
```

Figure 1.6 Pre-processor used to map routines to platform specific names.

1.2.2.1 Synchronization Primitives

One of the most critical areas of difference between the Linux and Windows operating systems is that of Synchronization primitives (Sync Objects). For example, the Windows operating system provides the facility of Critical Sections in addition to other synchronization primitives, whereas Linux has no equivalent for this functionality. Additionally, Windows uses single APIs (*WaitForSingleObject, WaitForMultipleObjects, and CloseHandle*) to operate on various *handles*.

In order to resolve part of the issue PAL implements Windows specific Sync Objects using *POSIX* synchronization primitives. Using this approach, corresponding Windows routines on Linux behave similar to Windows APIs allowing the original code to remain unmodified and for the details of the exact implementation to be hidden in the PAL library. For example, mutexes are used to provide the synchronization component for the routines implementing a critical section on Linux. Extra logic of reference count is added to allow the same thread to reenter critical sections without the need of recursive locks. Figure 1.6 below details this approach. A similar approach is used for Events where semaphores are used to emulate the behavior of Events on Windows.

In order to provide a single API like *WaitForSingleObject* or *CloseHandle* for various objects, PAL implements a unique nested class called PAL_*PtrMan*. PAL_PtrMan is intended to provide a single interface for all the various Windows specific *handles* implemented in PAL. All APIs used to create, delete or manipulate various objects accept and return PAL_PtrMan instance. Internally PAL_PtrMan maintains a pointer to the object it encapsulates for example a mutex pointer in case of a critical section instance returned by *PAL_initializeCriticalSection*. Various PAL_PtrMan methods distinguish the encapsulated pointer by using the PAL_TYPE_OF_PTR enum. Create and initialize methods of various objects update the class member `ptr_type` with an appropriately value of PAL_TYPE_OF_PTR. All subsequent calls to various APIs use this class member to determine which POSIX/Linux specific API to call for proper results. Figure 1.8 provides additional details.

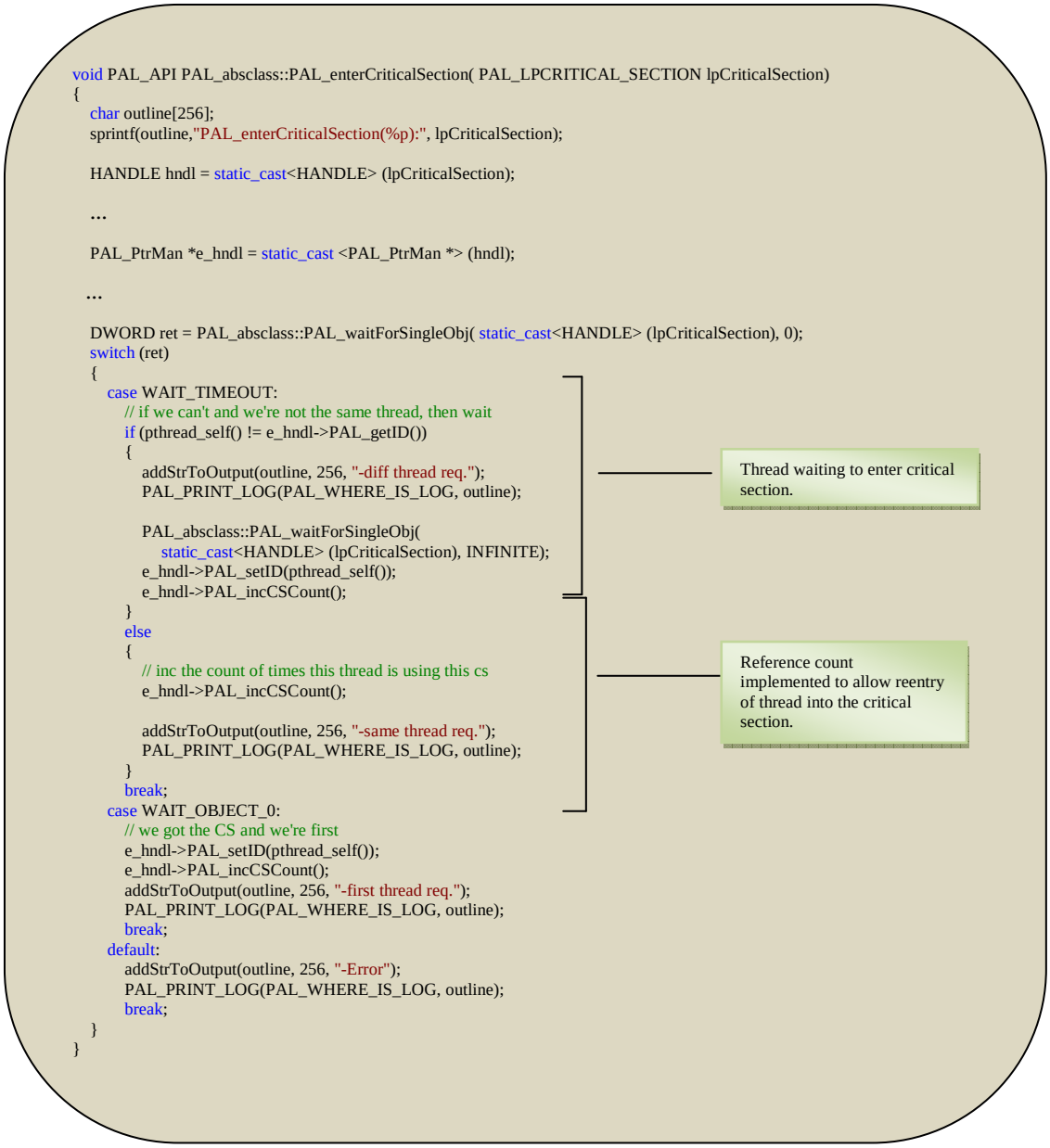


Figure 1.7 Example of PAL using Linux primitives to provide Windows synchronization functionality

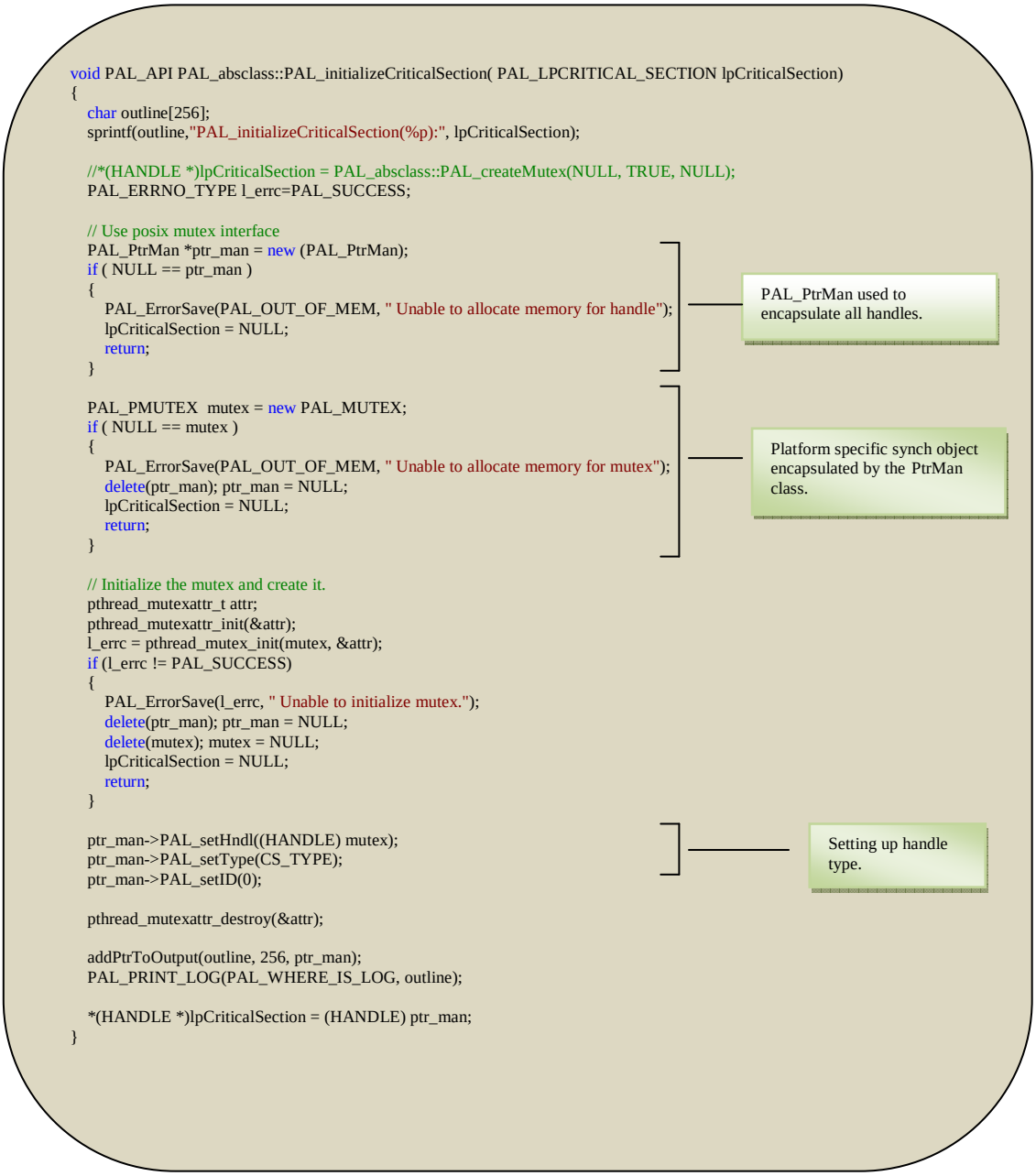


Figure 1.8 Implementation of Synch object handles in PAL

1.2.2.2 Name Space Resolution

All bridge routines used by PAL are implemented within a class as static functions. The primary benefit of this approach is to take advantage of C++ class structure as an abstract container for a logical grouping of unique identifiers. Thus, the methods implemented by PAL not only have global scope but they also help avoid namespace conflicts. Usage of class as opposed to C++ namespace is more appropriate as it lends to good object oriented design principles. Globally scoped methods also allow the abstraction layer to not require object instantiation and further no memory is used for pointers. Additionally, PAL uses private definition of methods to prevent the compiler from generating code, like the copy constructor, keeping the memory usage to the minimal. At the file level, all PAL specific files are prefixed with the string 'PAL_' to further avoid naming conflicts within the build structure.

1.2.3 Non-standard code and Compiler incompatibilities

There are several differences between Microsoft Visual C++ compiler and C++ GNU compiler which need to be handled properly for the HLX application. These include areas like template classes and exception handling. In the majority of cases, C++ standards were used to resolve these discrepancies. As an example differences in the exception thrown by the base class destructor and that thrown by (or not thrown at all) by the derived class destructor causes the build to fail. In order to get around this problem, destructor had to be added to the derived class, when appropriate, and key word throw() was added to the newly added destructor and all virtual methods throwing exceptions. Figure 1.9 details this approach.

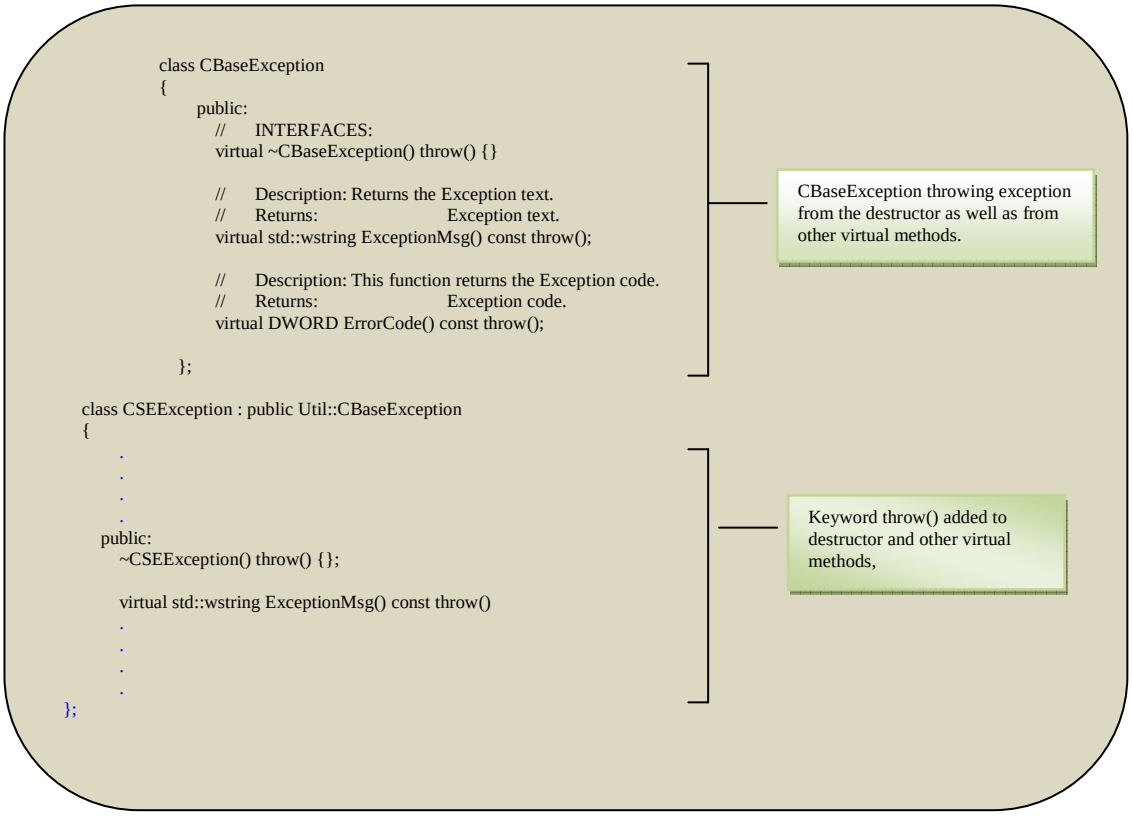


Figure 1.9 Compiler differences in exception handling.

1.3 Build System

The HLX application originally implied Visual Studio specific build process to build various projects constituting the application. Newer versions of Visual Studio implement XML based project files only hence Linux specific build process had to be separated from the Windows build setup.

In order to allow different projects to build independently each directory in the Linux build holds a unique Makefile. Each Makefile sets project specific values for certain variables which are then used by the main Makefile to build the project. Figure 1.10 lists the project specific Makefile and figure 1.11 details the main Makefile responsible for compiling and archiving the projects. Once all the projects are compiled and archived the daemon can be built by building the daemon target in the HOBART_APP/source/app directory

```
OS = linux

HOBART_APP ?= $(HOME)/projects/Hobart/shared_code/Application
export HOBART_APP
export HOBART_OUT_DIR=$(HOBART_APP)/output


###################################
#make vars
###################################

#
#Vars for Source and object files
#
SUB_DIRS := app comm core db EvolutionRTC/MessageCenter EvolutionRTC HFC util

C_SRCS :=
CPP_SRCS :=

###################################
# makefile template
###################################

include $(HOBART_APP)/hobart.mk
```

Figure 1.10 Project specific Makefile

```
including OUT_SUB_LIBS and OUT_OBJS)
#
OBJS = $(C_SRCS:.c=.o) $(CPP_SRCS:.cpp=.oo)
ifdef TARGET_LIB
   # When TARGET_LIB is defined, it's the root Makefile
   # ..... to do .....
endif

TARGET_BASENAME := $(subst /,_, $(subst $(HOBART_APP)/,, $(CURDIR)))
SUB_LIBS_BASENAME := $(addprefix $(TARGET_BASENAME)_, $(SUB_DIRS))
SUB_LIBS := $(addsuffix .a, $(SUB_LIBS_BASENAME))
TARGET := $(addsuffix .a, $(TARGET_BASENAME))
OUT_DIR = $(HOBART_OUT_DIR)
OUT_SUB_LIBS = $(addprefix $(OUT_DIR)/,$(SUB_LIBS))
OUT_OBJS = $(addprefix $(OUT_DIR)/,$(OBJS))
OUT_TARGET = $(addprefix $(OUT_DIR)/,$(TARGET))

INC_DIRS := $(INC_DIRS) \
            -I/usr/local/include/freetype2
#----------------------------------
#linux specific vars
#C Compiler vars
#
CC = gcc
CLINK = gcc
#
#C++ Compiler vars
#
# let gcc decide on the compiler to invoke as that is it's purpose
CXX = gcc
# g++ is the linker, not ar. ar is an archiver
CXX_LNK = g++
# don't put '-c' as a CXX_FLAG as '-c' is actually not a flag but the
# compiler action itself and these flags will need to be duplicated
# if we ever build a dynamic library during linking
CXX_FLAGS = -O0 -g -Wall -Wextra -DLINUX_FIX -D__STDC_FORMAT_MACROS#-Werror #-libstdc++ -fno-implicit-
templates -DDONT_INCLUDE_TEMPLATE_CODE #-fno-implicit-inline-templates
CXX_LFLAGS = $(CXX_FLAGS) -lrt `curl-config --libs` -lrt /usr/local/lib/libfreetype.a /usr/local/lib/libfreetype.so
CXX_FOR_TESTING_FLG = -E
AR = ar
AR_FLAGS = rcs
#Target defs
all: $(OUT_DIR) $(OUT_TARGET)
rootlib:
subdirs:
#
#build all sub dirs
#
all_dirs:
            @for dir in $(SUB_DIRS); do cd $$dir; $(MAKE) all; cd ..; done
# output directory
$(OUT_DIR):
            @if test -d $@; then echo "exist"; else mkdir $@; fi
# Link target library (***** WARNING: Next step: detect it's library or execution file)
$(OUT_TARGET): $(OUT_SUB_LIBS) $(OUT_OBJS)
ifneq ($(strip $(OUT_OBJS)), )
            @echo Linking library $(notdir $@)
            @$(AR) $(AR_FLAGS) $(OUT_TARGET) $(OUT_OBJS)
            @echo ----------
endif
```

Figure 1.11 Main Makefile

```
# Sub-directory libraries
.PHONY: $(SUB_DIRS)
$(SUB_DIRS):
	$(MAKE) -C $@
$(OUT_SUB_LIBS): $(SUB_DIRS)
#c build object files
$(OUT_DIR)/%.o : %.c
	@echo compiling $<
	@$(CC) -c $< $(INC_DIRS) -o $@
.c.o:
	@echo compiling $<
	@$(CC) -c $< $(INC_DIRS) -o $@
#c++ build object files
$(OUT_DIR)/%.oo : %.cpp
	@echo compiling $<
	$(CXX) $(CXX_FLAGS) -c $< $(INC_DIRS) -o $@
.cpp.o:
	@echo compiling file $<
	$(CXX) $(CXX_FLAGS) -c $< $(INC_DIRS) -o $@
#cleanup all output of this makefile
clean:
	@echo cleaning directory...
	@for dir in $(SUB_DIRS); do cd $$dir; $(MAKE) clean; cd ..; done
	@rm -f $(OUT_OBJS)
	@rm -f $(OUT_TARGET)
	@echo Done!
# create an exe to test that this library will link properly
# you will need to fill in the ARCHIVES linked in the
# projects local Makefile
build:
	@for dir in $(SUB_DIRS); do cd $$dir; $(MAKE) testbuild; cd ..; done
testbuild: $(OUT_TARGET)
	g++ $(CXX_LFLAGS) \
	   $(OUT_OBJS) \
	   -Wl,--start-group \
	   $(OUT_DIR)/PAL_PAL_main.a \
	   $(ARCHIVES) \
	   -Wl,--end-group \
	   -o main.exe
##################################
# Testing purpose
##################################
test:
	@echo test build
	@echo ----- Current path -----
	@echo $(CURDIR)
	@echo ----- Output path -----
	@echo $(OUT_DIR)
	@echo ----- Current relative path -----
	@echo $(CUR_RELATIVE_DIR)
	@echo ----- Target Basename -----
	@echo $(TARGET_BASENAME)
	@echo ----- Out OBJS -----
	@echo $(OUT_OBJS)
	@echo ----- Sub libs -----
	@echo $(SUB_LIBS)
	@echo ----- Out Sub LIBS -----
	@echo $(OUT_SUB_LIBS)
	@echo ----- Target -----
	@echo $(TARGET)
	@echo ----- HOBART_APP -----
	@echo $(HOBART_APP)
	@echo ------INC_DIRS -----
	@echo $(INC_DIRS)
```

Figure 1.11Main Make file continued…

1.4 Assumptions

The code will run on X86 based 32 bit processor.
The code will be buildable on XPe and Linux platform.
Compiler used to implement build the code will be ANSI/ISO C99 compliant.

Sized type will be assumed to follow 32bit programming model ILP32

Sized Type Name	Expected Size
int, unsigned int	32 bits
Long	32 bits
Long long	Not used
short, unsigned short	16 bits
char, unsigned char	8 bits